

[0:00–0:20 | Title]

“Good morning. I’m Samsu Sempena from the University of Sydney, and this is joint work with Zhuohan Cui. Our paper characterizes what speculative decoding actually does on consumer-class GPUs — the kind of hardware a practitioner has under their desk, not in a data centre.”

Cue: give the one-line thesis up front so the audience has a hook — “the short version is that the *policy ranking* travels between GPUs, but the *speedup* does not.”

Housekeeping: confirm the clicker works, and check whether the chair wants questions during or after.

[0:20–1:05 | Motivation]

Frame the bottleneck physically: every token costs a complete pass over the target model's weights, so on a single stream you move gigabytes of weights per token while the GPU's compute units sit mostly idle. That is why batch-1 interactive decoding is memory-bandwidth bound.

Then pivot: “speculative decoding is the well-known answer, and the literature reports two- to five-times speedups. But look at where those numbers come from — A100s, H100s, B200s. The practitioner running a 3B or 4B model on one RTX card has no reliable guidance.”

Point at the four questions on the right — these are exactly the knobs someone has to set on day one — and say we will answer all four with measurements.

Do not over-explain memory bandwidth; one sentence is enough.

[1:05–2:00 | Method recap]

Keep this brisk — most of the room knows the algorithm. Walk the strip once: the draft proposes four tokens; the target checks all four in a single pass; here it agrees on the first two, disagrees on the third, so everything from the mismatch onward is thrown away and the target supplies one corrected token. Net yield: three committed tokens for one verification step.

Then land on the equation, because it is the analytical spine of every result that follows: “the cost in the numerator grows linearly with k — you run the draft model k times. The payoff is τ , and τ saturates, because once acceptance drops off the extra proposals are wasted work. The theory already tells you there is a crossover. Our contribution is showing *where* it sits on real consumer hardware — and it sits much lower than people assume.”

Mention the correctness guarantee once. It pre-empts a question, and it makes the quality result later more interesting.

[2:00–2:40 | Gap and RQs]

Be explicit about the contribution type: “we are not proposing a faster drafter. We are asking whether the existing, unmodified draft–verify loop delivers on the hardware most people actually own, and answering it with the statistical care that a deployment decision deserves.”

Read RQ1 and RQ4 aloud; gesture at RQ2 and RQ3. RQ4 is the one they will remember, so give it a beat: “and the fourth question is the one that surprised us.”

If running long: skip the three gaps entirely, go straight to the RQs. Saves about 20 seconds.

[2:40–3:40 | Protocol]

The message here is *pairing*. Every speculative run is compared against a baseline that used the same prompts, the same sample IDs, the same length limits and the same sampling regime — so speedup is computed per sample, not as a ratio of two independent averages. That is what makes the paired *t*-tests later legitimate.

Note the deliberate choice of same-family pairings: matching tokenizers means no vocabulary-alignment overhead contaminating the measurement. Cross-family pairing is a different study.

On hardware, flag the counter-intuitive line: the laptop 5090 is the *newer* architecture — Blackwell versus Ada — but it has lower memory bandwidth and roughly a third of the power budget. Plant that seed now; it pays off at RQ4.

Be honest about the seed budget: full grid at seed 42, repeats only at $k=4$. Somebody will ask.

[3:40–4:55 | RQ1 headline]

This is the good-news slide — deliver it confidently, then immediately complicate it.

Lead with the tiles: “just over three times on Qwen2.5, just under three on Qwen3, both at deterministic $k=4$. Eleven of our twelve RTX4090 configurations beat their baseline.”

Then the hook into RQ2: “but notice which column the best numbers are in. It is the *smallest* k we tested. Every step toward a longer proposal costs us speed. And in the bottom-right cell — Qwen3, stochastic, $k=16$ — speculative decoding is actually *slower* than just running the target model normally.”

Pointer discipline: touch three cells only — the two peaks and the 0.9719. Do not read the table out.

[4:55–5:55 | RQ2 the inflection]

The analytically interesting result — slow down here.

“Our hypothesis going in — H2 — was the textbook one: $k=8$ should beat $k=4$, and $k=16$ might start to lose. That is wrong on this hardware. All four curves fall monotonically from the smallest k we tested. The inflection has already happened before $k=8$.”

Explain *why* with the equation from earlier: the effective block size does go up — you really do commit more tokens per verify step — but sub-linearly, while drafting cost goes up linearly. Acceptance is only 34% at $k=4$ and collapses to 13% at $k=16$, so most of those extra draft steps are pure waste.

The token-timing numbers are the practitioner’s punchline: at $k=16$ you have nearly tripled time-to-first-token. For an interactive application that is the worst possible place to spend latency — the user sees it directly.

Implication to state: on this class of hardware, tune k downward — and $k=4$ may not even be the floor.

[5:55–6:30 | RQ3 regime]

Short slide — the least surprising result, so move through it.

The mechanism is worth saying out loud: “when both draft and target decode greedily, the draft’s proposal is compared against a deterministic target continuation, so agreement is high. Turn sampling on for both and you introduce two independent sources of randomness; acceptance drops from about 34% to 27%, and you lose roughly 13% of your speedup.”

State the caveat rather than hiding it: at $k=16$ Qwen2.5 actually reverses, marginally. Small effect, but we report it — the preference is a strong tendency, not a law.

If running long: first slide to cut. Say “deterministic wins at matched k , by a margin that shrinks as k grows” and click through.

[6:30–7:50 | RQ4 portability — the core result]

This is the slide the talk exists for. Pause before starting it.

“We took the identical code, the identical models, the identical frozen manifests, and moved to an RTX5090 laptop — a *newer* architecture. Every single configuration is now slower than simply running the target model autoregressively. The best case, deterministic $k=4$, gives 0.67 — we have made inference about one and a half times slower.”

Then the diagnostic point, which is what makes this a finding rather than an anecdote: “acceptance is not the problem. At $k=4$ it is identical — 0.336 against 0.335. At $k=8$ and $k=16$ the laptop actually accepts *more* — 0.25 against 0.24, and 0.18 against 0.13. So the speculative algorithm is doing at least as well there as on the desktop, and it is still slower everywhere. What changed is the runtime economics: the draft model’s per-step cost relative to the target’s verification cost is unfavourable on this stack, and speculative decoding needs that ratio to be small.”

Land the deployment rule: “speedup is a property of the complete model–hardware–runtime stack. Not of the algorithm, and certainly not of the GPU generation — the newer card lost.”

Then the constructive half: the *ranking* did transfer. Low k beats high k , deterministic beats stochastic, on both devices. Policy exploration on one machine is transferable; absolute claims are not.

[RQ4, continued — questions to expect]

“Did you isolate the cause?” No, and say so. Our candidate explanation is host-side orchestration and graph-capture behaviour — see the CUDA-graph note in the paper — but we did not measure it, so we report it as context, not as a cause. Say this before someone asks.

“Is the laptop card simply weaker?” Partly — lower bandwidth, roughly a third of the power budget. But S is a *ratio* against that same device’s own autoregressive baseline, so a uniformly slower card should not by itself push the ratio below one. The backup hardware slide has the numbers.

“Would a better runtime fix it?” Very likely, and that is precisely the point: the fix lives in the stack, so the stack is what you have to measure.

[7:50–8:40 | Evidence quality]

Purpose: pre-empt the methodological objections, and show we are not over-claiming.

“Five of six RTX4090 configurations are significant after Holm–Bonferroni correction, with large effect sizes. The sixth — stochastic $k=16$ — is not significant, and its effect size is slightly negative, exactly consistent with its sub-baseline mean. We report it as a null, not as noise.”

The dispersion point matters more than the p -values: at $k=16$ the standard deviation across prompts is roughly half the mean. A single reported average speedup hides an enormous spread — your slowest prompts may see no benefit at all. This is one of the things the literature routinely omits.

On the chart: repeat seeds at $k=4$. Deterministic runs move by at most 1.5% between seeds; stochastic by up to 5%.

Say the limitation before a reviewer does: we only have repeat seeds at $k=4$. We do not claim seed stability at $k=8$ or $k=16$.

[8:40–9:25 | Quality trade-off]

Deliver this as an honest complication of our own headline number.

“Earlier I said the peak was deterministic $k=4$. On summarization that is also our *worst* quality point — ROUGE-L falls from 19.6 to 11.5, and BLEU and BERTScore move the same way. Three independent metrics agreeing means this is not a metric artefact.”

The direction is the interesting part: quality *recovers* as k increases, the opposite direction to speed. The two axes genuinely trade off across our grid, so the operating point is a choice, not a default.

Expected question — be ready: “isn’t speculative decoding supposed to be distribution-preserving?”

Answer: exactly, which is why this is worth reporting. The guarantee is about the sampling distribution under the idealized scheme; what we measure is an end-to-end implementation on a bounded generation budget, where accepted-prefix commitment and length limits interact. We flag it as an observed effect in our implementation and do not claim it refutes the theory. Do not oversell this — it is a limitation we are surfacing.

Note the chart indexes all three metrics to the baseline so they share one axis; the absolute values are in the table beside it.

[9:25–9:55 | Scorecard and limits]

Move quickly — this slide is mostly for the record and for the reviewers in the room. Do not read every bullet.

Highlight two things. First, H2: “we predicted $k=8$ would beat $k=4$ and it does not — the overhead inflection is earlier than the folklore suggests.” Second, the limitations column exists because we want the claim bounded: one runtime stack, one model family, one benchmark suite.

The CUDA-graph bullet is the intellectually honest one — we saw a difference in capture behaviour between run families, it is a plausible contributor to the RTX5090 result, but we did not isolate it, so we refuse to call it the cause.

Timing checkpoint: you should be at about 9:30 here. If you are past 10:30, skip straight to the takeaways.

[9:55–10:05 | The one line]

Pause. Let it sit for two seconds. This is the sentence you want people repeating to a colleague afterwards. Add nothing to it — click through.

[10:05–10:40 | Close]

Four sentences, one per point, then stop. Do not re-explain — the audience has seen all the evidence.

Rule one: short proposals. Rule two: greedy if you can afford it. Rule three — emphasize this one with your voice: measure on the machine you will actually deploy on, because we have a concrete counter-example where a newer GPU turned a three-times win into a one-and-a-half-times loss. Rule four: report the spread and the quality, because a mean speedup on its own is not a deployment decision.

Close on future work in one breath, then: “Thank you — I’m happy to take questions.”

Use if asked for the full Qwen3 numbers, or for TTFT/TPOT detail.

Qwen2.5 accepts more than Qwen3 at every k — $\alpha = 0.42$ vs. 0.34 at $k=4$, and effective block size 3.76 vs. 1.96 at $k=16$. That is why Qwen2.5 never falls below baseline on the RTX4090 while Qwen3 does. Likely a smaller capability gap between the 3B target and its 0.5B draft than between the 4B target and its 0.6B draft.

The six RTX5090-laptop numbers behind the RQ4 chart. Note B_{eff} is identical to the RTX4090 at every k (1.32/1.85/1.96 against 1.33/1.86/1.96): the speculative loop is committing the same number of tokens per verification step on both devices, and is still slower. That is the cleanest single piece of evidence that the loss is in execution cost.

The most useful backup slide for the RQ4 discussion. If someone challenges the RTX5090 result — “did you misconfigure it?” — this is the answer: acceptance is a model-level quantity, it is unchanged at $k=4$ and actually *better* at higher k , so the speculative loop is doing at least as well as on the desktop. The loss is in per-step execution cost, not in the algorithm.

Reference slide for methodology questions.

The obvious question after RQ4 is “is the 5090 laptop just a weaker card?” Partly — lower bandwidth, far lower sustained power. But that affects the autoregressive baseline too, and S is a *ratio* against that same device’s baseline. A uniformly slower card should not by itself flip the ratio below 1. That is why we point at the draft/target cost ratio and the runtime path rather than at raw card performance.